СЕМИНАР

Инструменты промышленной разработки

Конспект и рассказ · v2 (с поправками)

16 сентября 2026 · 19:45 — 21:03

Ведущий: Никита Шевердов · Курс MIT Missing Semester

Продолжительность: 1 ч 18 мин · ~108 участников

Первый семинар курса «Инструменты промышленной разработки» — практический разбор терминала, текстовых редакторов, отладчика PDB, потоков и bash-скриптов. Версия с поправками и дополнительными примерами.

Содержание

1. О курсе и философия
2. Основы терминала
3. Текстовые редакторы: nano и vim
4. Альтернативные терминалы и пакетные менеджеры
5. Python-дебаггер PDB
6. Потоки: stdin, stdout, stderr
7. Bash-скрипты: shebang и chmod
8. Ключевые решения и договорённости
9. Задачи и обещания
10. Вопросы и ответы
11. Поправки к сказанному на семинаре

Философия курса

Курс основан на MIT Missing Semester и закрывает пробелы в практических навыках. Оценка — среднее арифметическое из 5 домашних заданий. Семинарских оценок нет. Курс несложно закрыть на хорошую оценку.

1. О курсе и философия

В computer science часто бывает так: вам рассказывают математическую и теоретическую базу, а потом вы приходите на работу — и не понимаете, что такое коммит. Курс Missing Semester призван закрыть эти дыры: то, чем вы реально будете пользоваться и на ПМИ, и в индустрии.

Курс делится на три части: работа с компьютером, сборка, и AI/LLM (второй модуль). Никита работает в Яндексе, делает Алису AI почти год. Семинары записываются, их можно пересмотреть.

Главные принципы семинара:

- Глупых вопросов не существует — задавайте всё, что не понимаете
- Никакого буллинга за «глупые» вопросы — это критически осуждается
- Курс не про то, чтобы задушнить — это повайбовать и узнать новое
- Чем больше страдаете сейчас, тем легче будет в бою
- Обсуждайте задачи с людьми, а не с ChatGPT — придёте к более самостоятельной истине

> «Я супер за то, чтобы вы как можно больше страдали, потому что вы пока студенты. Если вам будет легко и вы будете закрывать все домашки через ChatGPT — вы ничего для себя не унесёте.»

2. Основы терминала

Терминал — та часть компьютера, которая есть всегда. В 70-е годы не было компьютерных мышей, и Shell был единственным способом управлять компьютером. SH и Bash можно рассматривать как языки программирования вокруг терминала.

Поправка: дефолтный shell на Mac

На семинаре сказано: «на всех маках дефолтно установлена SH-обёртка под названием The SH». Правильно: начиная с macOS Catalina (2019) дефолтный shell — ZSH (Z shell), не SH. SH всё ещё установлен в системе, но не используется по умолчанию.

Базовые команды:

Команда	Описание
`ls`	Показать содержимое текущей директории
`ls -la`	Расширенный листинг: права, владелец, скрытые файлы
`cd <dir>`	Перейти в директорию
`cd ..`	Вернуться на уровень вверх
`mkdir <name>`	Создать директорию
`pwd`	Показать абсолютный путь текущей директории
`cat <file>`	Вывести содержимое файла
`echo <text>`	Вывести текст на экран
`grep <pattern>`	Поиск по файлам
`grep -r <pattern> <dir>`	Рекурсивный поиск по всем файлам в директории
`head -n N <file>`	Первые N строк файла (по умолчанию 10)
`tail -n N <file>`	Последние N строк файла
`man <command>`	Руководство по команде (выход — Q)
`chmod +x <file>`	Добавить право исполнения
`history`	История последних выполненных команд
`Tab`	Автодополнение — ваш главный помощник
`Ctrl-C`	Послать сигнал прерывания (SIGINT) в процесс
`Ctrl-D`	Послать EOF — закрыть процесс типа exit

> **Поправка: Ctrl-D, не Ctrl-T**
>
> На семинаре сказано: «Ctrl-T закрывает окно полностью или выходит из процесса». Правильно: это Ctrl-D — он посылает EOF (end of file), что вызывает выход. Ctrl-T такого не делает.

Скрытые файлы:

Файлы, начинающиеся с точки (например, .env, .metadata), — скрытые. Это сделано для культуры: ключи, токены и конфигурация не попадают в общий доступ. ls без флагов их не показывает, нужен ls -la.

Пример .env файла для Telegram-бота:

```
BOT_TOKEN=123456789:ABCdefGhIJKlmNoPQRsTUVwxyz
```

Такой файл добавляют в .gitignore, чтобы не случайно не запушить токен.

3. Текстовые редакторы: nano и vim

Nano

Самый простой текстовый редактор, который есть на любом компьютере, любой системе. Если вы работаете на виртуальной машине где-то на сервере — вы не сможете открыть VS Code без танцев с бубном, а nano всегда под рукой.

Команда	Действие
`nano <file>`	Открыть/создать файл
`Ctrl-X`	Выход (предложит сохранить)
`Ctrl-O`	Сохранить
`Ctrl-K`	Вырезать строку
`Y + Enter`	Подтвердить сохранение при выходе

Vim

Vim — гораздо более продвинутый текстовый редактор с несколькими режимами. Появился из ещё более древнего редактора vi. Главное, что нужно уметь — из него выйти.

Режимы: Normal (просмотр) → нажать i → Insert (ввод текста) → Esc → обратно в Normal → :wq (write + quit) для сохранения и выхода, либо :q для выхода без сохранения, либо :q! для принудительного выхода без сохранения.

> **Поправка: значения :w и :q**
>
> На семинаре перепутано: «W это write, а Q это сохранить». Правильно: :w = write (сохранить), :q = quit (выйти). :wq = сохранить и выйти. :q! = выйти без сохранения.

Дополнительные команды vim:

Команда	Действие
`dd`	Удалить текущую строку
`yy`	Скопировать текущую строку (yank)
`p`	Вставить после курсора (paste)
`u`	Отменить (undo)
`Ctrl-r`	Повторить (redo)
`/text`	Поиск текста (n — следующее, N — предыдущее)
`gg`	Перейти в начало файла
`G`	Перейти в конец файла

> «Я никогда в жизни не видел более счастливых программистов, чем те, которые настроили себе Vim. Примерно 4 человека из аудитории останутся на нём до конца жизни. Остальным это больше никогда не пригодится.»

4. Альтернативные терминалы и пакетные менеджеры

Fish (friendly interactive shell)

Более красивый терминал, который запоминает ваши команды и помогает их дополнить. У Fish есть свой язык, несовместимый с bash-скриптами.

> **Поправка: расшифровка Fish**
>
> На семинаре: «SH это shell, а FI это что-нибудь красивое». Правильно: Fish = friendly interactive shell.

Warp

Open source терминал, более красивый. В нём можно командовать мышкой, есть AI-агенты. Тот же терминал, просто удобнее. Установка на Mac: brew install warp. Очень рекомендуется.

Пакетные менеджеры:

Менеджер	Платформа	Описание
`apt`	Debian/Ubuntu Linux	Стандартный пакетный менеджер (apt install <package>)
`brew`	macOS (и Linux)	Удобнее и красивее; brew install <package>
`sudo`	—	Команда для запуска с правами администратора (substitut

> **Поправка: apt не для Windows**
>
> На семинаре: «apt — пакетный менеджер для Linux и для Windows, наверное, тоже». Правильно: apt только для Debian/Ubuntu Linux. На Windows его нет.

5. Python-дебаггер PDB

PDB — Python Debugger, встроенный отладчик. То же, что красные флажки в PyCharm, но из терминала. PyCharm — сахарная оболочка над PDB: под капотом в код вставляются инъекции для отладки. VS Code требует танцев с бубном, а PDB есть у всех.

Запуск:

```
python -m pdb <file.py>
```

Флаг -m означает mode — PDB это имя режима. У команды python есть куча флагов, про каждый нужно читать отдельно (man python или python --help).

Команды PDB:

Команда	Действие
`b <func/line>`	Поставить breakpoint (по имени функции или номеру строки)
`c`	Continue — выполнение до следующего breakpoint
`n`	Next — выполнить следующую строку (без захода в функции)
`s`	Step — выполнить следующую строку (с заходом в функции)
`l`	List — показать ~11 строк вокруг текущей позиции
`ll`	Longlist — показать всю функцию целиком
`p <var>`	Print — вывести значение переменной
`a`	Args — показать аргументы текущей функции
`w`	Where — показать stack trace
`q`	Quit — выйти из отладчика

> **Поправка: l vs ll**
>
> На семинаре: «одна L показывает файл целиком, а две L — вокруг строки». Правильно: наоборот — l показывает ~11 строк вокруг текущей позиции, а ll (longlist) показывает всю функцию целиком.

Прикладной пример из реальной разработки:

Рекламный движок Яндекса — монолит с миллионами строк кода, которому ~16 лет. Вы меняете что-то про ставки в рекламном аукционе, тесты падают. Как среди миллионов строк понять, почему? Запускаете через дебаггер, расставляете брейкпоинты в ключевых местах, делаете continue и смотрите состояние системы. Находите: вот в этом файле система была в норме, а здесь — всё сломалось.

PDB позволяет остановиться на исключении и исследовать состояние: переменные, stack trace, аргументы функций — без принтов. Claude тоже умеет это делать, но хотя бы разочек в жизни нужно проделать руками.

Пример сессии PDB:

```
$ python -m pdb checkout.py
> /workspace/checkout.py(2)<module>()
> -> items = [...]
(Pdb) b total
Breakpoint 1 at /workspace/checkout.py:7
(Pdb) c
> /workspace/checkout.py(7)total()
-> result = 0
(Pdb) n
> /workspace/checkout.py(9)total()
-> for item in items:
(Pdb) p item, result
('mouse', 890)
(Pdb) q
```

6. Потоки: stdin, stdout, stderr

У любой программы есть потоки: один входной и как минимум два выходных. Их разделили, чтобы вычленять ошибки и складывать их в отдельные файлы.

Поток	№	Назначение
`stdin`	0	Стандартный ввод (input)
`stdout`	1	Стандартный вывод (обычные print)
`stderr`	2	Вывод ошибок (exceptions, warnings)

Перенаправление потоков:

Оператор	Действие
`> file`	Перезаписать файл выводом (стирает старое содержимое)
`>> file`	Дописать в конец файла
`< file`	Подать содержимое файла на stdin
`|`	Пайп: передать stdout одной команды на stdin другой
`1> file`	Перенаправить stdout в файл
`2> file`	Перенаправить stderr в файл
`2>&1`	Перенаправить stderr туда же, куда идёт stdout
`&&`	И: выполнить вторую команду, если первая успешна
`||`	Или: выполнить вторую команду независимо

Примеры:

```
# Записать абсолютный путь в файл, затем вернуться назад
cd data/row && pwd > answer.txt && cd ../..

# Раздельный вывод: stdout и stderr в разные файлы
python processor.py 1> stdout.log 2> stderr.log

# Достать вторую строку из файла через пайп
head -n 2 .metadata | tail -n 1

# Подать файл на stdin, результат записать в другой файл
python counter.py < input.txt > answer.txt

# Перенаправить и stdout, и stderr в один файл
python app.py > all.log 2>&1

# Рекурсивный поиск слова во всех файлах проекта
grep -r "import" .
```

Пайп (|) — это труба, по которой вы льёте данные. Выход одной команды становится входом другой. Например, head -n 2 берёт первые 2 строки, а tail -n 1

от результата берёт последнюю — так получается вторая строка файла.

tail -n 100 — типичная команда для чтения логов. Когда сервис сломался и логов 100 ГБ, вы не хотите читать их все — вы берёте последние 100 строк.

> **Поправка: head по умолчанию выводит 10 строк**
>
> На семинаре: «head дефолтно выводит первые пять, что ли, строк». Правильно: head по умолчанию выводит 10 строк, не 5.

7. Bash-скрипты: shebang и chmod

Bash-скрипты написаны на том же языке, что и команды в терминале. У вас уже есть вся база для их написания. В рамках курса их нужно будет написать один раз в жизни.

Создание bash-скрипта:

1. Создайте файл hello.sh
2. Первая строка — shebang: #!/bin/bash (без неё скрипт не работает)
3. Команда echo "Hello from Shell" — вывод текста
4. Сохраните через nano (Ctrl-X, Y, Enter)
5. Запуск: ./hello.sh → permission denied
6. chmod +x hello.sh — добавить право исполнения
7. ./hello.sh → Hello from Shell ✓

Пример скрипта:

```
#!/bin/bash
echo "Hello from Shell"
```

Пример посложнее — скрипт с переменной и условием:

```
#!/bin/bash
NAME=$1
if [ -z "$NAME" ]; then
    echo "Usage: $0 <name>"
    exit 1
fi
echo "Hello, $NAME!"
```

Запуск: ./hello.sh Вася → Hello, Вася!

> «Пообещайте мне, пожалуйста, больше никогда ничего в жизни на bash не писать. Если ваша программа больше двух строчек и вы хотите писать циклы или функции — напишите на Python. Вы знаете Python. Всё, что можно сделать на bash, можно сделать на Python тоже.»

История shell'ов:

- SH (70-е) — оригинальный shell
- Bash (Bourne Again Shell) — от фамилии Born
- ZSH — через год после Bash, похожий язык
- Fish (friendly interactive shell) — новый терминал с подсветкой и своим языком

8. Ключевые решения и договорённости

Тестирующая система

Доступна до завтрашнего вечера, потом закроется. Каждому участнику — свой Docker-контейнер с Linux в Яндекс.Облаке. ~108 человек зашли.

Домашние задания

Появятся на днях. Оценка — среднее арифметическое из 5 домашних работ. Семинарских оценок нет, на сессии тоже.

План занятий

Будет опубликован, если студентам это необходимо. Перед лекциями и семинарами темы анонсируются.

LLM и AI

Вторая часть курса (конец второго модуля). Не про «как пользоваться ChatGPT», а про то, как это устроено внутри и как применять в работе.

Linux

Рекомендуется устанавливать как можно раньше и переводить учебные процессы на него. Начиная со второго курса Windows не будет.

Тестирующая система — не на оценку

Задачи в арене — для практики. Подсказки уменьшают очки. Домашка будет в другом месте.

9. Задачи и обещания

Задача	Ответственный	Статус
Выгрузить материалы семинара (команды, объяснения) в общий конспект	Никита	В процессе
Опубликовать план всех занятий курса	Никита и Саша	Если будет интерес

10. Вопросы и ответы

Q: Когда первое домашнее задание?

A: На днях. Конкретную дату назвать нельзя.

Q: Будут ли оценки за семинары?

A: Нет. Оценка — среднее арифметическое из 5 домашних работ.

Q: Когда начнём говорить про LLM?

A: Скорее во втором модуле, под конец.

Q: Есть ли план всех занятий?

A: План есть у Никиты и Саши. Если студентам прям очень надо — опубликуют. Перед лекциями и семинарами темы анонсируются.

Q: Можно ли поставить виртуальную машину с Linux на ноутбук с Windows?

A: Да, можно. Но лучше как можно раньше переводить все учебные процессы на Linux полностью, а не в рамках виртуальной машины — так удобнее жить.

Q: Чем отличается cat от p (print) в PDB?

A: p — команда внутри Python-дебаггера. cat — команда терминала (ZSH). Разные контексты, разные команды.

Q: Чем отличается > от >>?

A: > перезаписывает файл с нуля. >> дописывает в конец файла.

Q: Как выбрать конкретную строку из файла по индексу?

A: head -n N | tail -n 1 — берём первые N строк, затем последнюю из них. Или tail -n 1 для последней строки файла. Это нишевая потребность; чаще нужен tail -n 100 для чтения последних строк логов.

11. Поправки к сказанному на семинаре

Сводка всех фактических неточностей, допущенных в ходе семинара, с исправлениями:

1. Дефолтный shell на Mac

Сказано: «на всех маках дефолтно установлена SH-обёртка The SH».

Правильно: начиная с macOS Catalina (2019) дефолтный shell — ZSH (Z shell). SH установлен, но не используется по умолчанию.

2. Ctrl-T vs Ctrl-D

Сказано: «Ctrl-T закрывает окно или выходит из процесса типа exit».

Правильно: это Ctrl-D — он посылает EOF (end of file). Ctrl-T такого не делает.

3. Значения :w и :q в vim

Сказано: «W это write, а Q это сохранить» (перепутано).

Правильно: :w = write (сохранить), :q = quit (выйти). :wq = сохранить и выйти. :q! = выйти без сохранения.

4. head: дефолтное число строк

Сказано: «head дефолтно выводит первые пять строк».

Правильно: head по умолчанию выводит 10 строк, не 5.

5. PDB: l vs ll

Сказано: «одна L показывает файл целиком, а две L — вокруг строки».

Правильно: наоборот. l показывает ~11 строк вокруг текущей позиции, ll (longlist) — всю функцию целиком.

6. apt и Windows

Сказано: «apt — пакетный менеджер для Linux и для Windows, наверное, тоже».

Правильно: apt только для Debian/Ubuntu Linux. На Windows его нет.

7. Расшифровка Fish

Сказано: «SH это shell, а FI это что-нибудь красивое».

Правильно: Fish = friendly interactive shell.

Конспект составлен на основе транскрипта записи семинара от 16 сентября 2026 г. Курс «Инструменты промышленной разработки», ВШЭ ПМИ. Основан на MIT Missing Semester. Версия 2 — с поправками и дополнительными примерами.